

Enabling Team Collaboration



Ned Bellavance

HashiCorp Ambassador

@ned1313 | nedinthecloud.com



Module Overview



Explore state data contents

- terraform state command

State backends

Migrating state data



Globomantics Environment



GLOBOMANTICS

Collaborate with networking team

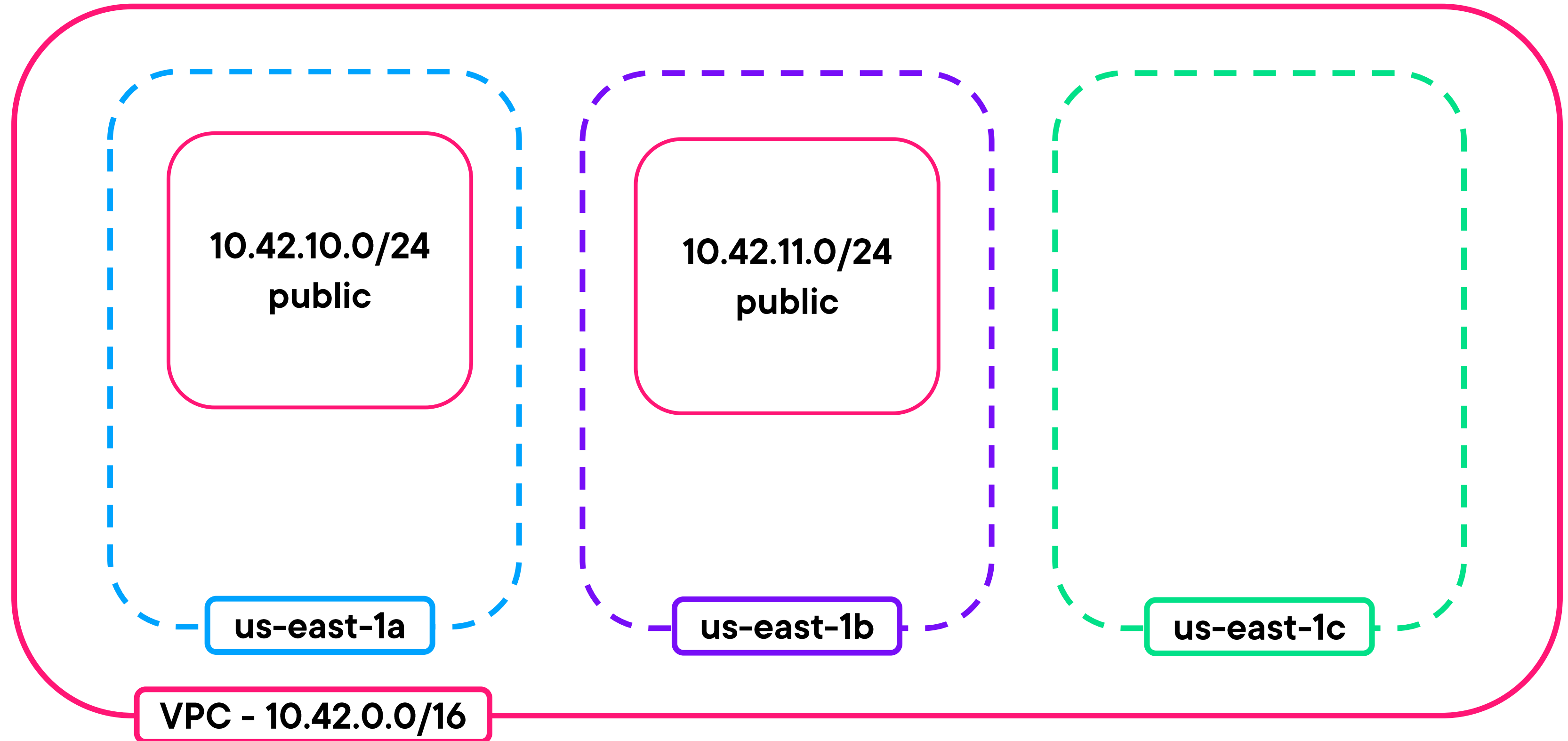
Share information with application teams

Enable state data access

Leverage Terraform Cloud for remote state



Existing Environment



Local State Data

terraform.tfstate

```
{
  "resources": [
    {
      "mode": "managed",
      "type": "aws_vpc",
      "name": "main",
      "instances": [
        {
          "id": "vpc-123654789"
        }
      ]
    }
  ]
}
```

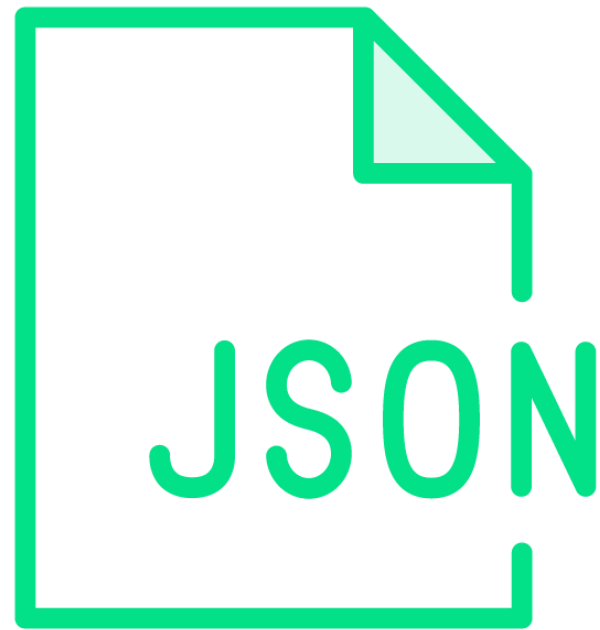
main.tf

```
module "main" {
  source      = "terraform-aws-..."
  version     = "5.0.0"

  cidr_block = var.vpc_cidr
}
```



Terraform State Data



Mapping of configuration to actual resources

Stored in JSON (don't touch!)

State contents

- Metadata
- Resources, data sources, and outputs



```
{
  ...
  "resources": [
    {
      "mode": "data",
      "type": "aws_availability_zones",
      "name": "available",
      "instances": [{
        "id": "us-east-1",
        ...
      }]
    }
  ]
}
```

◀ **Resource list**

◀ **Mode set to data for a data source**



```
{
  ...
  "resources": [
    {
      "module": "module.main",
      "mode": "managed",
      "type": "aws_vpc",
      "name": "this",
      "instances": [{
        "id": "vpc-123654789",
        ...
      }]
    }
  ]
}
```

◀ **Resource list**

◀ **Module address**

◀ **Mode set to managed for a resource**

◀ **Type set to "aws_vpc"**

◀ **Name set to "this"**

◀ **Instances at the resource address**

◀ **ID set to actual resource**




```
{
  ...
  "outputs": {
    "vpc_id": {
      "value": "vpc-123654789",
      "type": "string"
    }
  }
}
```

◀ **Outputs map**

◀ **Output name**

◀ **Value of output**

◀ **Type of output**

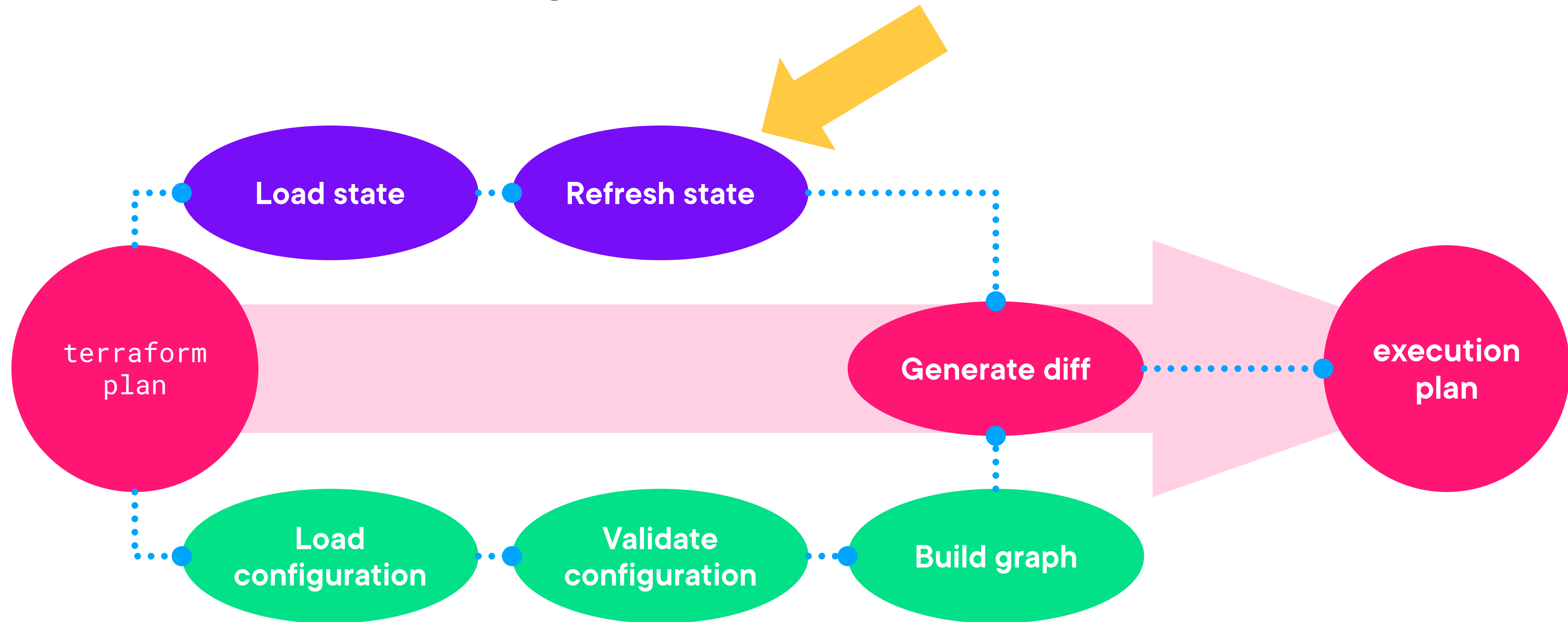




Interacting with State Data



Terraform Planning Process



Refreshing State Data

Refresh state data based on managed resources

terraform refresh

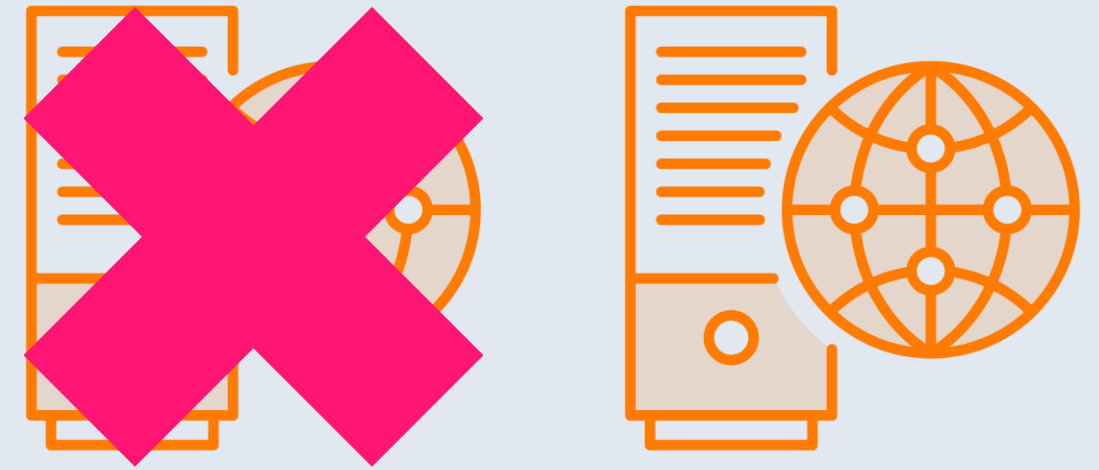
Refresh state data only as part of execution plan

terraform plan -refresh-only

terraform apply -refresh-only



Recreating Resources



Mark a resources for recreation in state

```
terraform taint ADDR
```

```
terraform taint aws_instance.web
```

Replace a resource as part of the execution plan

```
terraform plan -replace="aws_instance.web"
```

```
terraform apply -replace="aws_instance.web"
```


Moving Resources

```
terraform state mv aws_vpc.main aws_vpc.web
```

main.tf

```
resource "aws_vpc" "main" {  
  cidr_block      = var.vpc_cidr  
  enable_dns_support = true  
  
  tags = {  
    Name = var.vpc_name  
  }  
}
```

main.tf

```
resource "aws_vpc" "web" {  
  cidr_block      = var.vpc_cidr  
  enable_dns_support = true  
  
  tags = {  
    Name = var.vpc_name  
  }  
}
```



Moving Resources

```
# Move resources from old to new address
terraform state mv OLD_ADDR NEW_ADDR
terraform state mv aws_vpc.main aws_vpc.web
```

```
# Moved block
```

```
moved {
  from = OLD_ADDR
  to   = NEW_ADDR
}
```

```
moved {
  from = aws_vpc.main
  to   = aws_vpc.web
}
```



Removing Resources

```
# Remove the resource entry at the given address  
terraform state rm ADDR  
terraform state rm aws_vpc.main  
  
# Also must remove from configuration
```



Other State Commands

List all resources and data sources

```
terraform state list
```

List all attributes of a single object

```
terraform state show ADDR
```

```
terraform state show aws_vpc.main
```

Send a copy of state data to stdout

```
terraform state pull
```

Push state data to a path

```
terraform state push PATH
```





Terraform State Backends



Backend Options

{JSON}

Local backend

- Default setting

Remote backend

- S3, Azure Storage, Google Cloud Storage
- Terraform Cloud and Consul

Features

- Locking
- Workspaces

Data encryption

Access control



Backend Block

backend.tf

```
terraform {  
  backend "backend_type" {  
    IDENTIFIER = VALUE  
  }  
}
```



Backend Block

backend.tf

```
terraform {  
  
  backend "s3" {  
  
    bucket      = "globo-state-12345"  
    key         = "dev/network/terraform.tfstate"  
    dynamodb_table = "globo-state-12345"  
    region      = "us-east-1"  
    access_key   = "HGT657UHT56786YUHT6"  
    secret_key   = "ytgvbJU&*ijhgt6%TYUijhy"  
  
  }  
  
}
```



Backend Block

```
backend.tf
```

```
terraform {  
  
    backend "s3" {  
  
        bucket      = "globo-state-12345"  
        key          = "dev/network/terraform.tfstate"  
        dynamodb_table = "globo-state-12345"  
  
    }  
  
}
```



```
terraform {  
    backend "s3" {}  
}
```

Partial Configuration

Using in-line settings

```
terraform init -backend-config="bucket=globo-state-12345"
```

Using a backend config file

```
terraform init -backend-config="backend-settings.txt"
```





Backend Planning



Globomantics Environment



GLOBOMANTICS

Collaborate with networking team

Share information with application teams

Enable state data access

Leverage Terraform Cloud for remote state



Cloud Block

backend.tf

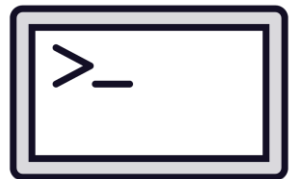
```
terraform {  
  cloud {  
    organization = "globo-deep-dive"  
  
    workspaces {  
      name = "web-network-dev"  
    }  
  }  
}
```



Migrating State Data



Update configuration with backend or cloud block



Run terraform init with any required flags



Confirm state data migration



Module Summary



Terraform state is kinda important

Declarative state management

Remote state for security and collaboration



Up Next:

Building a CI/CD Workflow

